
Rapport de PCL2

Semaine 05 à semaine 15

Rédigé par

CLOÉE HAREAU
STEVEN TRINH
ADRIEN BASCHUNG
WENJIA TANG

Table des matières

1	Gestion de projet	3
1.1	Généralités	3
1.2	Répartition du nombre d'heures.	3
1.3	Contexte / hypothèses de départ : Matrice SWOT	4
1.4	Déroulement du projet	4
2	Travail réalisé	6
2.1	Asr.java	6
2.2	AsrCreator.java	6
2.2.1	Convention de registres	6
2.2.2	Utilisation tas	6
2.3	Package visual	7
2.3.1	ExitTrappedException et ExitTrapper	7
2.3.2	Launcher	7
2.3.3	OutputParser	7
3	Difficultés rencontrées	8
3.1	Accès aux Tds	8
3.2	Debug	8
3.3	Bug VisUAL	8
3.4	Print	9
3.5	Accès aux variables	9
3.6	Copie des variables composées : Array, Record, String	9
4	Annexes	10
4.1	Schémas de traduction	10
4.1.1	Record	10
4.1.2	For	11
4.1.3	Appels de fonctions	11
4.1.4	Identificateur	12
4.1.5	Array	12
4.1.6	Accès à un élément d'un array	16
4.1.7	Accès à un field d'un record	16
4.2	Jeux d'essai	17
4.2.1	Tri fusion	17
4.2.2	Test copie d'un array d'arrays	18
4.2.3	Test copie d'un array de records	18
4.3	Comptes-rendus des réunions	19

1 Gestion de projet

1.1 Généralités

Auteurs de ce cahier des charges / groupe d'expression du besoin

Nom / mail :	Qualité / rôle
Steven TRINH steven.trinh@telecomnancy.eu	Chef de Projet / Responsable de projet
Cloée HAREAU cloee.hareau@telecomnancy.eu	Projet / Membre de l'équipe
Adrien BASCHUNG adrien.baschung@telecomnancy.eu	Projet / Membre de l'équipe
Wenjia TANG Wenjia.Tang@telecomnancy.eu	Projet / Membre de l'équipe
Suzanne COLLIN suzanne.collin@univ-lorraine.fr	Commanditaire

Les réunions ont été planifiées en fonction des emplois du temps de chacun, favorisant les week-ends et les vacances scolaires. Ainsi, la plupart des réunions se sont déroulées à distance sur la plateforme Discord.

Les différents documents du projet étaient partagés sur le Gitlab de TELECOM Nancy, très utilisé pour le partage de fichiers de code, les documents de la gestion de projet étaient eux partagés sur un Google Drive (et sont aussi disponibles sur le Gitlab).

1.2 Répartition du nombre d'heures.

N'ayant pas noté au fur et à mesure les heures de travail, le temps noté ci-dessous est approximatif.

Noms	Temps passé sur le rapport	Temps passé sur le code
Steven TRINH	3h	42h
Adrien BASCHUNG	3h	35h
Wenjia TANG	7h	32h
Cloée HAREAU	14h	21h

1.3 Contexte / hypothèses de départ : Matrice SWOT

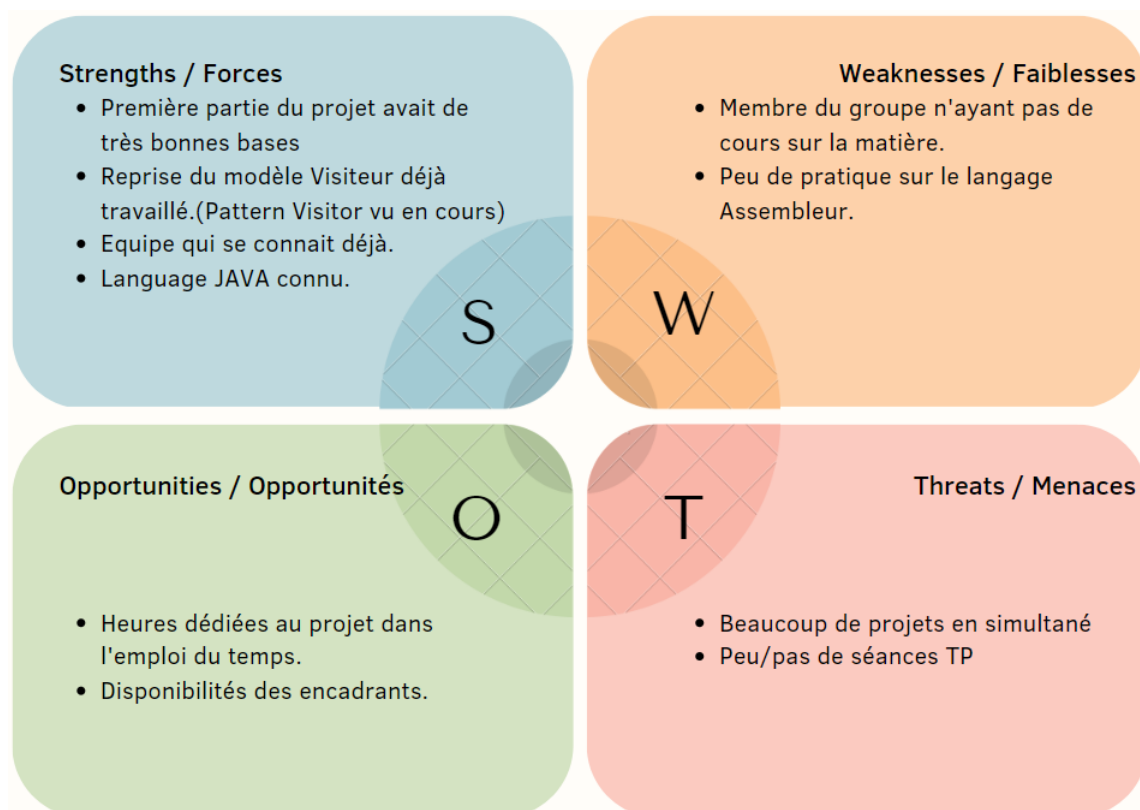


FIGURE 1.1 – Matrice SWOT

1.4 Déroulement du projet

Le projet prend place du mois de février 2023 jusqu'au mois d'avril 2023.

Après avoir construit une grammaire LL(1), écrit un code permettant de consulter les arbres syntaxiques des programmes écrits à l'aide du langage et créé un arbre syntaxique abstrait ainsi que des tables de symboles permettant de mettre en avant les erreurs sémantiques. Le projet consistait désormais à créer le compilateur qui permettrait la traduction du code Tiger en langage Assembleur.

Dans l'optique d'une meilleure organisation du projet, nous avons séparé différentes parties de programmation du projet pour nous les répartir équitablement.

On retrouve ici le diagramme de Gantt pour la répartition des tâches.

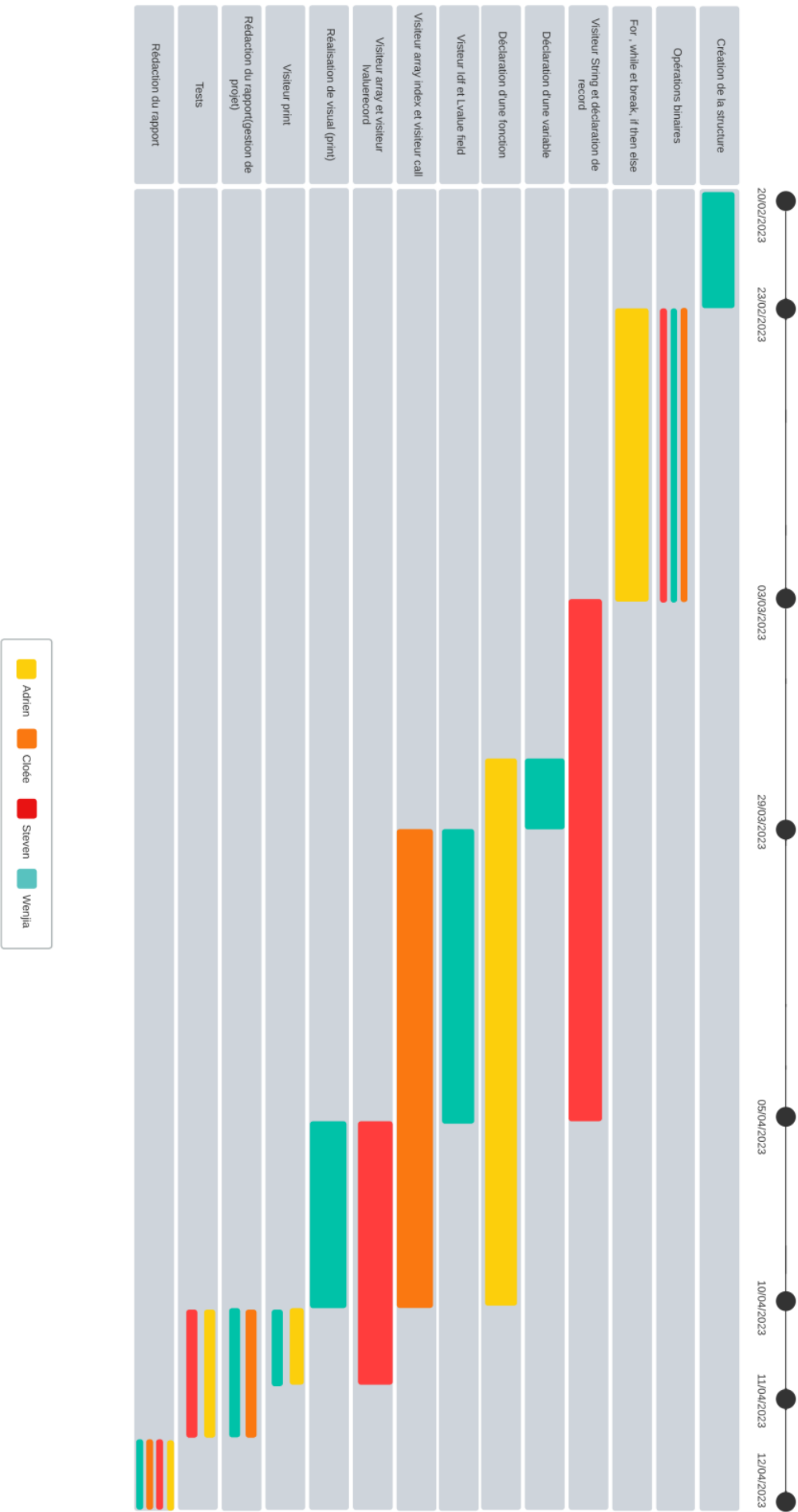


FIGURE 1.2 – Diagramme de Gantt

2 Travail réalisé

Dans la deuxième partie du projet de Compilation, notre objectif principal est de traduire le code Tiger en langage assembleur *ARM UAL*.

Pour cela, nous avons créé le package **assembleur** qui contient le package **visual**, ainsi que les fichiers **Asr.java** et **AsrCreator.java**. Le package **visual** est conçu pour permettre l'exécution directe de code assembleur en Java, en utilisant la bibliothèque **visual_headless.jar**. La classe **Asr.java** a pour fonction d'écrire du code ARM dans un fichier **out.S**. Enfin, le fichier **AsrCreator.java** permet de parcourir les différents nœuds du code Tiger et d'appeler des méthodes de la classe **Asr** pour effectuer la traduction automatique. Chaque section sera détaillée plus en profondeur pour plus de clarté.

2.1 Asr.java

La classe **Asr** a été créée dans le but de factoriser les codes assembleurs en une seule classe, ce qui permet d'améliorer la lisibilité et la clarté du code dans la classe **AsrCreator**.

Cette classe contient une liste de chaînes de caractères représentant les codes assembleurs. Cette liste est complétée lors de la visite de différents nœuds du programme à traduire. Ensuite, nous utilisons la méthode **File.write** en Java pour écrire les codes assembleurs dans un fichier de sortie appelé **out.S**.

2.2 AsrCreator.java

La classe **AsrCreator** occupe la place centrale dans notre projet de traduction de code Tiger en langage assembleur *ARM UAL*. Son rôle est de parcourir l'arbre syntaxique de notre programme et d'ajouter les codes assembleurs correspondants à la liste de la classe **Asr** pour chaque nœud.

2.2.1 Convention de registres

Pour éviter les conflits, nous avons choisi d'utiliser **R0** pour enregistrer la valeur de retour, tandis que les registres **R1** à **R6** sont disponibles pour effectuer des calculs.

Le registre **R9** est utilisé pour enregistrer l'ancienne adresse de Base Pointer lorsqu'il est nécessaire de déplacer la base pointer pour récupérer des valeurs dans la pile.

R10 est utilisé pour enregistrer l'adresse de retour, tandis que **R11** représente l'adresse de la première case vide du tas.

Enfin, **R12** et **R13** sont conventionnellement utilisés pour représenter le base pointer et le stack pointer. Dans notre cas, la pile est ascendante et le tas est descendant.

2.2.2 Utilisation tas

Nous avons stocké toutes les structures composées dans le tas :

- Record : Nous avons stocké les éléments à la suite, dans l'ordre dans lequel ils apparaissent dans le programme, lors de la déclaration du type.

- Array : Nous avons stocké tous les éléments à la suite, et nous réservons la première case afin d’y stocker la taille du tableau.
- String : Nous avons stocké les éléments à la suite, un caractère par octet. Ainsi une case de 4 octets pourra contenir 4 caractères au maximum.

Cette organisation permet de représenter chaque variable avec une taille de 4 octets : un int est codé par 4 octets, et les autres variables sont représenté par l’adresse du tas contenant les données de la variable, adresse qui est codée sur 4 octets. De plus, l’utilisation du tas est presque semblable à l’utilisation de la pile, on utilise le registre r11 pour situer la première case vide au-dessus de son sommet. Le tas commence au même endroit que la pile, mais est descendante, contrairement à la pile qui est ascendante.

2.3 Package visual

L’émulateur *VisUAL* ne supporte pas la manipulation et l’affichage de chaîne de caractères. Ainsi, nous sommes obligés de passer par du code *Java* pour réaliser cette fonctionnalité, c’est également la raison pour laquelle nous avons créé le package visual.

Dans ce package, nous avons 4 classes en total, ce sont :

- classe **ExitTrappedException**,
- classe **ExitTrapper**,
- classe **Launcher**,
- classe **OutputParser**.

2.3.1 ExitTrappedException et ExitTrapper

En effet, ces deux classes sont utilisées pour redéfinir les comportements de la méthode *System.exit()* afin d’empêcher l’émulateur VisUAL d’arrêter brutalement l’exécution du programme en cas d’erreur. La classe **VisualExitManager** permet de capturer les appels à la méthode *System.exit()* et de lancer une exception correspondante, tandis que la classe **VisualSecurityManager** sert à contrôler les permissions d’accès des classes chargées dynamiquement par l’émulateur. Ces deux classes sont nécessaires pour que l’exécution du programme puisse se dérouler correctement dans l’émulateur VisUAL.

2.3.2 Launcher

La classe **Launcher** a pour fonction de faire exécuter le code assembleur généré par la classe **Asr-Creator** en utilisant la version sans interface graphique de l’émulateur VisUAL. Elle permet également de générer un fichier de log au format XML contenant le contenu de la mémoire à chaque adresse contenue dans la variable *print_out*. Pour cela, il est nécessaire de configurer le logging pour écrire dans le fichier le contenu de *print_out* à chaque breakpoint. Dans notre projet, le breakpoint est la ligne où nous avons utilisé l’instruction *str* pour enregistrer notre chaîne de caractères à imprimer sur le terminal.

2.3.3 OutputParser

La classe **OutputParser** a pour objectif d’extraire à partir des fichiers de logs générés par la classe **Launcher** le contenu de l’espace mémoire *print_out*, de décoder les chaînes de caractères qu’il contient et de les afficher sur le terminal. Pour cela, elle lit le fichier de log XML, extrait les valeurs hexadécimales de l’espace mémoire *print_out*, les convertit en chaînes de caractères ASCII et les affiche sur le terminal. Cette classe est donc un outil pratique pour vérifier si les entiers que nous avons essayé d’imprimer dans notre programme Tiger ont bien été générés et affichés correctement.

3 Difficultés rencontrées

3.1 Accès aux Tds

Nous avons d'abord comme idée de récupérer les Tds dans l'ordre dans lequel ils apparaissent dans la liste. Le principe était que l'AST étant parcouru dans le même ordre dans l'analyse sémantique et dans la génération de code, nous devrions récupérer la bonne TDS en récupérant les TDS linéairement.

Cependant, nous avons omis le fait que pour effectuer l'analyse sémantique des fonctions mutuellement récursives, il nous avait fallu modifier l'ordre de parcours de la TDS, ce qui a rendu notre solution non valide. Nous avons donc dû modifier la méthode d'accès aux TDS.

Finalement, nous avons créé deux nouvelles classes :

- `LinkNodeTds` : S'occupe d'enregistrer un lien entre un nœud de l'AST et une TdS.
- `LinkList` : Représente une liste de liens, elle permet de renvoyer la TdS correspondante à un certain nœud.

Pour mettre cette solution en place, il nous a fallu créer un nouveau lien à chaque création de TdS dans le `TdsCreator` puis passer la `LinkList` en paramètre de notre créateur de `ArsCreator`, pour pouvoir facilement accéder aux tables de symboles.

3.2 Debug

Le debug de la génération de code était particulièrement difficile, car suivre l'exécution du langage assembleur demande beaucoup de temps et de concentration. Cependant, nous avons trouvé un outil qui a permis de simplifier ce debuggage : `VisUAL2`.

Il s'agit d'un outil semblable à `VisUAL`, mais dont l'affichage mémoire est simplifié. Il nous était plus facile de suivre l'exécution afin de comprendre l'origine des erreurs pour pouvoir les rectifier.

De plus, l'implémentation, bien que tardive, du print qui était l'un des derniers nœuds implémentés, a également facilité le debug grâce à la possibilité d'identifier plus facilement la zone du programme à partir de laquelle le comportement du programme divergeait du comportement attendu.

3.3 Bug VisUAL

Enfin, l'une des dernières difficultés était un problème avec le logiciel `VisUAL`. En effet, notre programme **AsrCreator** renvoie un texte à placer dans `VisUAL`, et `VisUAL` n'arrive pas à compiler certaines instructions, notamment l'instruction :

```
1 LDMFA    R12, PC
```

Étant dans l'impossibilité de résoudre ce problème, nous avons décidé de remplacer cette instruction par plusieurs d'entre-elles :

```
1 MOV      R2, SP
2 SUB      SP, SP, #4
3 LDR      R12, [SP, #0]
4 SUB      SP, SP, #4
5 LDR      PC, R2
```


3.4 Print

Effectivement, pour afficher un entier sur le terminal, il est nécessaire de convertir cet entier en son code ASCII. Cependant, nous avons plusieurs situations différentes pour effectuer cette conversion, ce qui a augmenté la difficulté pour trouver le bon ordre des breakpoints. Pour résoudre le problème, nous avons décidé de mettre une instruction *STR* pour enregistrer un **ETX** (end of text) à la fin d'un entier avant quitter la branche *print*. Ça nous permet de mettre un breakpoint unique à la fin de la branche *print* et de récupérer l'intégralité de la chaîne.

Pour éviter de multiplier les labels et retrouver plus facilement l'adresse de la chaîne de caractères à afficher, nous avons décidé de ne mettre qu'un seul label nommé *print_out*. Cependant, cette méthode peut poser des problèmes si plusieurs valeurs sont écrites dans la même adresse. Par exemple, si une chaîne de caractères "45" est enregistrée dans l'adresse *print_out*, puis une autre chaîne de caractères "5" est écrite dans la même adresse, la première valeur de "45" ("4") sera écrasée, mais pas la deuxième ("5"). Ainsi, lorsque nous essayons d'afficher "5", la chaîne de caractères "55" sera affichée. Pour résoudre ce problème, nous avons décidé de nettoyer la valeur de chaque case mémoire après avoir stocké l'indicateur de fin de chaîne de caractères *ETX*. Cela signifie que nous remettons la valeur de chaque case mémoire à 0 après avoir stocké l'indicateur *ETX*.

3.5 Accès aux variables

Pour faciliter l'accès aux variables, nous avons choisi d'organiser la pile de façon similaire quel que soit le type de bloc dans lequel nous entrons :

Dans le cas d'une fonction, nous devons empiler les paramètres avant de pouvoir empiler l'adresse de retour, nous avons donc choisi d'imiter cet ordre pour les nœuds *For* et *Let*, ainsi, les variables sont toujours empilées en premiers.

Ensuite, dans le cas des nœuds *For* et *Let*, il n'y a pas d'adresse de retour à empiler, et le chaînage statique et dynamique sont toujours égaux, nous avons donc choisi d'empiler pour ces deux nœuds uniquement un chaînage, qui fait à la fois office de chaînage statique et dynamique, à la suite des variables.

Finalement, pour rester cohérent dans la déclaration et l'appel de fonction, nous commençons par empiler le chaînage statique, dont l'adresse fera également office de Base Pointer, ce qui permettra d'effectuer exactement les mêmes opérations pour l'accès à une variable, quel que soit le type de bloc la définissant, de même pour remonter les chaînages statiques. Enfin, nous empilons le chaînage dynamique et l'adresse de retour à la suite.

3.6 Copie des variables composées : Array, Record, String

La déclaration de variables pose une ambiguïté pour les types stockés dans le tas (record, array et string). En effet, il nous faut déterminer si l'affectation se fait par référence ou par valeur (copie). Si le passage se fait par référence, alors pour le code en annexe 4.2.2, *board[1][1]* et *board[2][1]* pointeront vers la même case mémoire. Ainsi, on obtiendrait en sortie 1.

Après en avoir discuté avec notre commanditaire, il a été décidé de faire la déclaration de variables de type record par référence, mais par valeur (copie) pour les types array et string. Ainsi, il nous a donc fallu implémenter des fonctions permettant de copier un array et toutes ses ramifications de manière récursive (ex : pour *var IL := class[N] of eleve* où *eleve* est un record, il faudrait alors non seulement créer N records contenant les mêmes données que *eleve*, mais aussi les éventuels arrays contenus dans les fields de manière récursive).

4 Annexes

4.1 Schémas de traduction

4.1.1 Record

```
1 noeud LvalueRecord :
2     empiler_SP("R11") // On empile dans la pile l'adresse de la 1ere case du
      record situee dans le tas
3
4     generer_code(noeud_fieldList) // noeud de la liste des fields du record
5
6     depiler_SP("R0") // On recupere dans R0 l'adresse de la 1ere case du
      record situee dans le tas

1 noeud FieldList :
2     decrementer_HP(4*nb de fields) // On descend le HP a la prochaine case
      vide du tas en avance pour les cas ou pendant la generation des fields,
      on devra empiler dans le tas d'autres structures (pour un record de
      record par exemple)
3
4     Pour chaque noeud field dans listField, faire : // listField est une
      liste, et non un noeud
5         generer_code(field)

1 noeud Field :
2     generer_code(noeud_membreDroit) // R0 = resultat de l'expression (
      adresse dans le tas ou int)
3
4     //On lit le sommet de la pile qui contient l'adresse de la 1ere case du
      record dans le tas
5     // Le sommet de la pile sera toujours l'adresse du record car le noeud
      Field est necessairement la continuite du noeud LvalueRecord
6     lireVarReg("R1", "SP") // LDR R1, [SP]
7
8     // depl est l'indice du field dans le record lors de la declaration du
      type
9     // On stocke le resultat de l'expression dans le tas    l'emplacement
      correspondant pour le field
10    // Comme le tas est descendant, on se deplace de -depl
11    str("R0", "R1", -depl)
```

4.1.2 For

```
1 noeud For :
2     visiter noeud_DEBUT // On visite le noeud debut, valeur dans r0
3     empiler r0 // on empile la valeur du noeud_DEBUT, valeur initiale de la
    variable iteree du for
4     empiler r12 // on empile la valeur de l'ancien BP (Chainage)
5     deplacer r12,SP // On met a jour le nouveau BP
6     visiter noeud_FIN // On visite le noeud fin, valeur dans r0
7     empiler r0 // on empile la valeur limite du for
8
9     acceder r1,r12-1 // la variable est enregistree 1 case en dessous de BP,
    retour dans r1
10    acceder r0,r12+1 // la valeur finale est enregistree 1 case au dessus de
    BP, retour dans r0
11    comparer r0,r1
12    si r0 < r1 aller a 20
13
14    generer code noeud_BLOC
15
16    acceder r0,r12-1 // on accede a i
17    add r0,r0,#1
18    sauvegarder r0,r12-1 // on met a jour i
19    aller a 9
20    deplacer SP,r12 //On retourne au BP
21    depiler r12 // On remet l'ancien BP dans r12
22    SUB SP,SP,#4 // On depile la variable itarative
```

4.1.3 Appels de fonctions

```
1 noeud Call :
2     generer code noeud_PARAM; // on empile les parametres
3     generer code noeud_ID; // apres la visite du noeud R0 contient l'adresse
    qui pointe vers la declaration de la fonction
4
5     int nb_param = fct.getNumberOfParameters();
6
7     //Trouver le Chainage Statique
8     deplacer("R9", "R12"); // on deplace R12 dans R9
9     for (int i=0;i<(nb_imbri_actuel-nb_imbri_def);i++){
10         lireVarReg("R9","R9"); // LDR R9, [R9]
11     }
12     empiler_SP("R9"); // on empile le chainage statique
13
14     deplacer("LR", "PC");
15     deplacer("PC","R0"); // On se deplace vers la definition de la fonction
    ou le CD et l'@ret sont empiles. Puis on execute le code. Avant de
    revenir ici, on d pile le CD et l'@ret.
16
17     depiler_SP; // on depile le CS
18
19     for (int i=0;i<nb_param;i++) {
20         depiler_SP; // on depile tous les parametres.
21     }
```

4.1.4 Identificateur

```
1 noeud Idf :
2     visiterMembreName; // Recuperer le nom
3     asr.lireAdrBP("R9");//Enregistrer l'ancienne adresse du base pointeur
4
5     while (tdsCourant.existLocalVarFunc == false){
6         asr.lireValBP("R10",0);//Lire le chainage statique et l'enregistre
        dans R0
7         asr.positionnerBP("R10");//Positionner BP vers l'adresse du chainage
        statique
8         tdsCourant = tdsCourant.getParent();
9         }//Lorsque la variable ou la fonction cherchee n'est pas local,
        alors nous nous deplacons dans sa derniere imbrication en utilisant le
        chainage statique. Nous effectuons cet etape jusqu'a nous trouvons la
        variable ou la fonction dans une table de symbole.
10
11     getDeplacement; //Recuperer le deplcement dans la table de symbole
12     asr.incrementerBP(deplacement);// Positionner le base pointeur vers la
        premiere case memoire de la variable ou de la fonction
13     asr.mov("R10","R12");// Lire l'adresse du base pointeur et l'enregistrer
        dans R0
14     asr.lireVarReg("R0","R10");//Lire sa premiere case memoire.
15
16     asr.positionnerBP("R9");//Repositionner le base pointeur vers son
        ancienne adresse.
```

4.1.5 Array

```
1 noeud Array :
2     // R1 = taille du tableau sur laquelle on va iterer
3     // R2 = pointeur de chaque case de l'array dans le tas
4     // R3 = pointeur vers la structure A COPIER situee dans le tas
5
6     empiler_SP("R11") // On empile dans la pile l'adresse de la 1ere case de
        l'array situee dans le tas
7
8     generer_code(noeud_index) // R0 = taille du tableau
9
10    empiler_HP("R0") // On stocke dans la 1ere case de l'array sa taille
11
12    mov("R1", "R0") // R1 = taille du tableau
13    mov("R2", "R11") // R2 = pointeur de chaque case de l'array dans le tas
        (pointe actuellement vers la 2e case)
14
15    decrementer_HP(4*taille_array) // On descend le HP a la prochaine case
        vide du tas en avance pour les cas ou pendant la generation des elements,
        on devra empiler dans le tas d'autres structures (pour un array de
        record par exemple) (on utilise R1 ici)
16
17    empiler_SP("R1, R2")
18    type_element = generer_code(noeud_exprElement) // On genere le code
        correspondant aux elements de l'array (array of exprElement)
19    depiler_SP("R1, R2")
20
21    mov("R3", "R0") // R0 = resultat de l'expression (adresse dans le tas ou
        int)
22
23    Si les elements de array sont de type int, alors :
```

```

24     str("R3", "R2") // On stocke directement le int dans la case de l'
    element situee dans le tas
25     sinon : // l'array est compose de variables de type record, array ou
    string et R3 = pointeur vers la structure a copier situee dans le tas
26     empiler_SP("R1, R2, R3")
27     copierType(type_element)
28     depiler_SP("R1, R2, R3")
29
30     str("R0", "R2") // On stocke le resultat de la copie dans la case de
    l'element situee dans le tas
31
32     sub("R2", "R2", "#4") // On descend le pointeur d'element a la prochaine
    case du tas
33     // Ne pas decrementer R3 car on veut toujours pointer sur la structure a
    copier (racine de l'arborescence) : on effectue taille_array copies de
    la meme structure pointee par R3
34
35     sub("R1", "R1", "#1") // On decremente la taille de l'array pour iterer
    sur chaque case
36
37     Si R1 > 0, aller a ligne 23 // Si on a fait les taille_array copies
38
39     depiler_SP("R0") // On recupere dans R0 l'adresse de la 1ere case du
    array situee dans le tas

```

```

1  copyType(type_element) :
2      // Feuille
3      Si type_element est string :
4          copyString()
5      // Noeud
6      sinon :
7          Si type_element est array :
8              copyArray(type_element)
9          sinon : // Record
10             copyRecord(type_element)

```

```

1  copyArray(type_element) :
2      // Le sommet de la pile DOIT contenir le pointeur vers la structure a
    copier
3
4      // R1 = taille du tableau sur laquelle on va iterer
5      // R2 = pointeur de chaque case de l'array A REMPLIR dans le tas
6      // R3 = pointeur vers l'array A COPIER situee dans le tas
7
8      lireVarSP("R3") // On recupere le pointeur vers l'array a copier (R3 si
    appelant est noeud Array (initialisation), R4 si appelant est copyArray (
    iteration))
9      // R3 = 1ere case de l'array, donc la case de la longueur de l'array
10
11     empiler_SP("R11") // On empile dans la pile l'adresse de la 1ere case de
    l'array a remplir situee dans le tas
12
13     lireVarReg("R1", "R3") // R1 = taille du tableau
14     sub("R3", "R3", "#4") // R3 pointe sur le 1er element de l'array a
    copier
15
16     empiler_HP("R1") // On stocke dans la 1ere case de l'array sa taille
17
18     mov("R2", "R11") // R2 = pointeur de chaque case de l'array a remplir
    dans le tas (pointe actuellement vers la 2e case)
19

```

```

20     decrements_HP(4*taille_array) // On descend le HP a la prochaine case
    vide du tas en avance pour les cas ou pendant la generation des elements,
    on devra empiler dans le tas d'autres structures (pour un array de
    record par exemple) (on utilise R1 ici)

21
22     Si les elements de array sont de type int, alors : // Feuille
23         lireVarReg("R4", "R3") // On stocke dans R4 le int contenu dans la
    case de l'array a copier
24         str("R4", "R2") // On stocke directement le int dans la case de l'
    element a remplir situee dans le tas
25     sinon : // l'array est compose de variables de type record, array ou
    string et R3 = pointeur vers LE POINTEUR de la structure a copier situee
    dans le tas
26         empiler_SP("R1, R2, R3")
27         lireVarReg("R4", "R3") // R4 = pointeur vers la structure a copier
    situee dans le tas
28         empiler_SP("R4")
29         copierType(type_element)
30         decrements_SP(4) // R4 n'est pas utile donc on depile seulement
31         depiler_SP("R1, R2, R3")
32
33         str("R0", "R2") // On stocke le resultat de la copie dans la case de
    l'element situee dans le tas
34
35     sub("R2", "R2", "#4") // On descend le pointeur d'element a remplir a la
    prochaine case du tas
36     sub("R3", "R3", "#4") // On descend le pointeur d'element a copier a la
    prochaine case du tas
37
38     sub("R1", "R1", "#1") // On decremente la taille de l'array pour iterer
    sur chaque case
39
40     Si R1 > 0, aller a ligne 22 // Si on a fait les taille_array copies
41
42     depiler_SP("R0") // On recupere dans R0 l'adresse de la 1ere case du
    array situee dans le tas

```

```

1 copyRecord(type_element) :
2     // Le sommet de la pile DOIT contenir le pointeur vers la structure a
    copier
3
4     // R1 = pointeur de chaque case du record A REMPLIR dans le tas
5     // R2 = pointeur vers la structure A COPIER situee dans le tas
6
7     lireVarSP("R2") // On recupere le pointeur vers le record a copier (
    necessairement R4 empile par copyArray (iteration))
8     // R2 = 1ere case du record
9
10    empiler("R11") // Pour le retour
11    mov("R1", "R11") // R1 va pointer sur chaque element du record
12
13    decrements_HP(4*nb_fields)
14
15    Pour chaque field du record, faire :
16        Si field est de type int, alors : // Feuille
17            lireVarReg("R3", "R2") // On stocke dans R4 le int contenu dans
    la case du field a copier
18            str("R3", "R1") // On stocke directement le int dans la case du
    field a remplir situee dans le tas
19        sinon : // le field est de type record, array ou string et R3 =

```

```

20     pointeur vers LE POINTEUR de la structure a copier situee dans le tas
21     empiler_SP("R1", R2")
22     lireVarReg("R3", "R2") // R3 = pointeur vers la structure a
23     copier situee dans le tas
24     empiler_SP("R3")
25     copierType(type_element)
26     decrements_SP(4) // R3 n'est pas utile donc on depile seulement
27     depiler_SP("R1", R2")
28
29     str("R0", "R1") // On stocke le resultat de la copie dans la
30     case du field situee dans le tas
31
32     sub("R1", "R1", "#4") // On descend le pointeur du field a remplir a
33     la prochaine case du tas
34     sub("R2", "R2", "#4") // On descend le pointeur du field a copier a
35     la prochaine case du tas
36
37     depiler_SP("R0") // Retour du record rempli
38
39 // Copie en feuille de l'arborescence de la structure a copier
40 copyString() :
41     // Ici, on n'incremente pas le HP car on ne connait pas la taille du
42     string en avance
43     // On itere jusqu'au character de terminaison
44
45     // R1 = pointeur de chaque case du string A REMPLIR dans le tas
46     // R2 = registre contenant les chars a stocker dans le tas (= le contenu
47     de R1)
48
49     mov("R0", "R11") // Pour le retour de la string
50
51     lireVarSP("R1") // On recupere le pointeur vers la string a copier (
52     necessairement R4 empile par copyArray ou copyRecord (iteration))
53     // R1 = 1ere case du string
54
55     lireVarReg("R2", "R1") // R2 = chars courants de la string a copier (4
56     characters par registre car 1 character = 2 hexa)
57
58     empiler_HP("R2") // On stocke les 4 chars de la copie dans la case du
59     string a remplir
60
61     // En utilisant and("R3", "R2", "#0b11")
62     Si R2 pointe sur la fin du string, aller a ligne 24
63
64     charSuivant("R1") // On incremente le pointeur de string pour passer aux
65     4 chars suivants
66     Aller a ligne 11
67
68 //FIN

```

4.1.6 Accès à un élément d'un array

```
1 noeud LvalueIndex:
2     visiterMembreGauche; // De facon recursive, nous allons chercher l'@ de
   l'element, en commençant avec l'ID.
3     empiler_SP("R0"); // on sauvegarde l'adresse de l'array
4
5     visiterIndiceDroit; // la valeur de l'indice est maintenant dans R0
6
7     depiler_SP("R1");           // on recupere l'@ de l'array
8     lireVarReg("R3", "R1");     // R3 contient la taille de l'array
9     deplacer("R2", "R0");       // R2 contient l'indice
10
11     comparer("r3","r2"); // On compare la taille de l'array avec l'indice
   recherche car nous ne pouvons pas faire le controle semantique.
12     Si r2<=r3 {
13         multby4("R2"); // "LSL R2, R2, #2" Indice*taille
14         soustraire("R1","R1","R2"); // on descend dans le tas
15         Deplacer("R10","R1"); // on met l'adresse de l'array dans R10
16         lireVarReg("R0", "R10");
17     }
```

4.1.7 Accès à un field d'un record

```
1 noeud LvalueField:
2     visiterMembreGauche; //Chercher l'adresse de ce field lors de la viste
   de membre gauche, l'adresse sera enregistree dans le registre R10, de
   plus, nous recuperons son type qui est essentiel pour retrouver le
   deplacement dans notre table de symbole.
3     asr.lireAdrHP; // Enregistrer l'ancienne adresse du pointeur de tas avant
   de la deplacer.
4     positionnerHP; //En utilisant l'adresse recuperee depuis R10, nous
   positionnons notre pointeur de tas vers l'adresse du membre gauche.
5     visiterIdf; //Chercher le nom de type du field en visitant noeud Idf.
6     tds.getFieldDeplacement; //Chercher la valeur du deplacement dans notre
   table de symbol.
7     tds.getFieldType; // Retrouver le type du field dans la table de symbole
8     asr.decrmenterHP(deplacement); //Positionner le pointeur du tas vers le
   field
9     asr.mov("R10","R11"); // Recuperer l'adresse du field cherche.
10    asr.lireVarReg("R0","R10"); //Enregistrer la valeur du field dans R0
11    asr.positionnerHP("R9"); // Repositionner le pointeur du tas vers ou il
   etait
```


4.2 Jeux d'essai

4.2.1 Tri fusion

```
1 let
2   type list = {head : int, tail : list}
3   type coupleList = {L1 : list, L2: list}
4   var L := list{head = 5, tail = list{head = 2, tail = list{head = 3, tail
5     = list{head = 1, tail = list{head = 4,tail = list{head = 6, tail = nil
6     }}}}}}
7
8   var sortedList : list := list{head = 0, tail = nil}
9   var list_ptr : list := list{head = 0, tail = nil}
10
11   function separe(L : list) : coupleList =
12     if L = nil then(
13       coupleList{L1 = nil, L2 = nil}
14     )
15     else(
16       if L.tail = nil then (
17         coupleList{L1 = list{head = L.head, tail = nil}, L2 = nil}
18       )
19       else(
20         let
21           var couple := separe(L.tail.tail)
22           in
23             coupleList{L1 = list{head = L.head, tail = couple.L1},L2
24             = list{head=L.tail.head, tail = couple.L2}}
25         end
26       )
27     )
28
29   function fusion(couple : coupleList) : list =
30     if couple.L1 = nil then(
31       couple.L2
32     )
33     else (
34       if couple.L2 = nil then(
35         couple.L1
36       )
37       else(
38         if couple.L1.head < couple.L2.head then(
39           list{head = couple.L1.head, tail = fusion(coupleList{L1
40             = couple.L1.tail, L2 = couple.L2})}
41         )
42         else(
43           list{head = couple.L2.head, tail = fusion(coupleList{L1
44             = couple.L1, L2 = couple.L2.tail})}
45         )
46       )
47     )
48
49   function triFusion(L : list) : list =
50     if L = nil | L.tail = nil then(
51       L
52     )
53     else(
54       let
55         var couple := separe(L)
56       in
57         fusion(coupleList{L1 = triFusion(couple.L1), L2 = triFusion(
```

```

        couple.L2))
53         end
54     )
55 in
56     sortedList := triFusion(L);
57
58     list_ptr := sortedList;
59     while list_ptr <> nil do (
60         print(list_ptr.head);
61         list_ptr := list_ptr.tail
62     )
63 end

```

4.2.2 Test copie d'un array d'arrays

```

1 let
2     var N := 2
3
4     type intArray = array of int
5     type intArrayArray = array of intArray
6
7     var row := intArray[N] of 0
8     var board := intArrayArray[N] of row
9 in
10     print(board[1][1]);
11
12     board[1][1] := 1;
13     print(board[2][1])
14 end

```

4.2.3 Test copie d'un array de records

```

1 let
2     var N := 2
3
4     type student = {name : string, age : int}
5     type class = array of student
6
7     var steven := student{name = "Steven", age = 21}
8     var crew := class[N] of steven
9 in
10     crew[1].name := "Wenjia";
11
12     if crew[2].name = "Wenjia" then (
13         print(0)
14     );
15     if crew[2].name = "Steven" then (
16         print(1)
17     )
18 end

```

4.3 Comptes-rendus des réunions

Réunion du 23/02/2023
ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion Brainstorming	Lieu : Réunion en présentiel (TELECOM)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 23/02/2023 - 20h Durée : 1h

Ordre du jour

- Discuter du sujet, partager les idées sur l'écriture du code.
- Répartition du travail à réaliser.
- Valider la structure du projet.

Décisions

- Pour boucle for : ajout de la valeur finale dans la pile.
- Pour opérations de retour : le résultat est dans le registre R0.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Écriture des boucles while, for, break et if-then-else	Adrien BASCHUNG	03/03/2023 18 h 00	fichiers JAVA	Toute l'équipe
Écriture des Oérateurs binaires	Steven TRINH Wenjia TANG Cloée HAREAU	03/03/2023 18 h 00	fichiers JAVA	Toute l'équipe

Prochaine réunion : 03/03/2023

Réunion du 03/03/2023
ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 03/03/2022 - 21 h 30 Durée : 1 h

Ordre du jour

- Tester les opérateurs binaires.
- Vérification du travail déjà effectué.

Décisions

- Ordre de pile pour les définitions de fonction : CS - CD - @ret.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Visiteur String et déclaration de record	Steven TRINH	29/03/2023 18 h 00	Fichiers JAVA	Toute l'équipe
Déclaration d'une variable	Wenjia TANG	29/03/2023 18 h 00	Fichiers JAVA	Toute l'équipe
Déclaration d'une fonction	Adrien BASCHUNG	29/03/2023 18h00	Fichiers JAVA	Toute l'équipe
Visiteur array index et visiteur call	Cloée HAREAU	29/03/2023 18h00	Fichiers JAVA	Toute l'équipe

Prochaine réunion : 29/03/2023

Réunion du 30/10/2022
ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 29/03/2023 - 15 h Durée : 45 min

Ordre du jour

- Continuer les visiteurs, discuter des problèmes et partager des idées.
- Vérification du travail déjà effectué.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Continuer le travail commencé	Steven TRINH Adrien BASCHUNG Cloée HAREAU	05/04/2023 18 h 00	Codes JAVA	Toute l'équipe
Visteur Idf et Lvalue field	Wenjia TANG	05/04/2023 18 h 00	Code JAVA	Toute l'équipe

Prochaine réunion : 05/04/2023

Réunion du 05/04/2023
ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 05/04/2023 - 23 h Durée : 1 h

Ordre du jour

- Continuer les visiteurs, discuter des problèmes et partager des idées.
- Vérification du travail déjà effectué.
- Création du dépôt Leaf : pour le projet Latex.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Continuer le travail commencé	Adrien BASCHUNG Cloée HAREAU	10/04/2023 10 h	Codes JAVA	Toute l'équipe
Réalisation de visual (print)	Wenjia TANG	10/04/2023 10h00	Codes JAVA	Toute l'équipe
Visiteur array et visiteur lvaluerecord	Steven TRINH	10/04/2023 10h00	Codes JAVA	Toute l'équipe

Prochaine réunion : 10/04/2023

Réunion du 10/04/2023
ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 10/04/2023 - 14h Durée : 1h

Ordre du jour

- Continuer les visiteurs, discuter des problèmes et partager des idées.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Continuer le travail commencé	Steven TRINH	11/04/2023 10h00	Codes JAVA	Toute l'équipe
Visiteur print	Adrien BASCHUNG Wenjia TANG	11/04/2023 10h00	Codes JAVA	Toute l'équipe
Ecriture de la partie Gestion de projet du rappot	Cloée HAREAU	11/04/2023 10h00	Rapport LATEX	Toute l'équipe
Diagramme de Gantt	Wenjia TANG	11/04/2023 10h00	Image PNG	Toute l'équipe
Effectuer les tests terminaux	Adrien BASCHUNG Steven TRINH	11/04/2023 10h00	Code TIGER et JAVA	Toute l'équipe

Prochaine réunion : 11/04/2023

Réunion du 11/04/2023
ÉCRIT PAR CLOÉE HAREAU

Motif / type de réunion : Réunion d'avancement	Lieu : Réunion en distanciel (Discord)
Présents : Adrien BASCHUNG, Wenjia TANG, Steven TRINH, Cloée HAREAU	Date : 11/04/2023 - 18h30 Durée : 30min

Ordre du jour

— Vérification du travail effectué.

Todo List :

Description	Responsable	Délai	Livrable	Validé par
Continuer les tests terminaux	Tous	12/04/2023 17h00	Code TIGER et JAVA	Toute l'équipe
Terminer écriture du rapport	Tous	12/04/2023 17h00	Rapport LATEX	Toute l'équipe